

RESEARCH SEMINAR

SimToolReal

An Object-Centric Policy for
Zero-Shot Dexterous Tool Manipulation

Can one reusable controller
turn tool motion into robot skill?

Kushal Kedia* · Tyler Ga Wei Lum*

Jeannette Bohg · C. Karen Liu

Cornell University · Stanford University

RSS 2026 · Based on arXiv v2, 24 February 2026

Real



Everyday tools demand dexterous control

Brush (Zero-Shot)

Task: Sweep Forward



Task: Sweep Right



WHAT TO WATCH

The tool extends what the robot can do.

The hand must establish a grasp, change the tool orientation, and maintain contact.

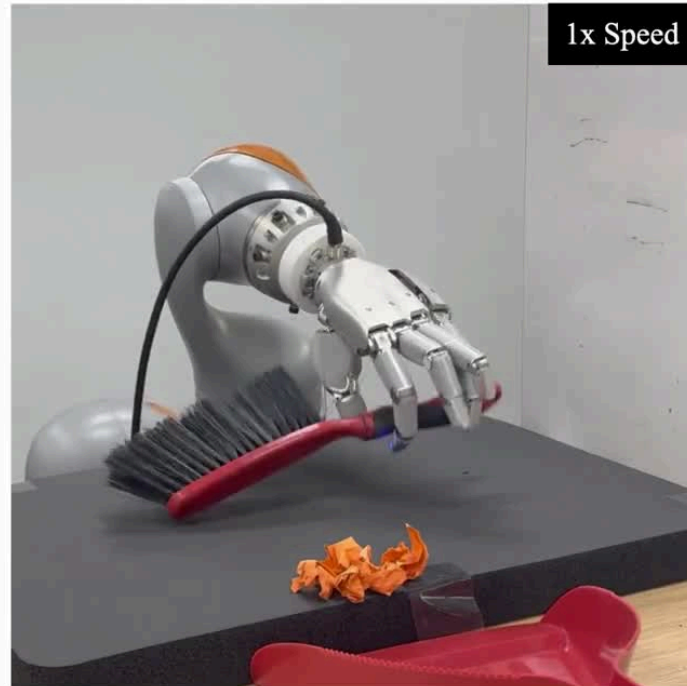
Question: how should a robot acquire this capability?

SimToolReal project: brush manipulation · 1× speed · silent excerpt

[Original project video ↗](#)

Tool use combines several contact problems

Grasp



WHAT TO WATCH

Acquire: grasp a thin tool on a flat surface.

Reorient: move it within the fingers.

Interact: resist external forces without losing the tool.

SimToolReal project: why tool manipulation is hard · 1× speed · silent excerpt

[Original project video ↗](#)

How could a robot learn to use a new tool?

Would you collect more data,
build a task simulator,
or change **what the policy learns**?

The design choice is both a learning method and an interface between the task and the robot.

Several routes place the burden in different places

Route	What supplies the behavior?	Main burden for dexterous tools
Robot demonstrations / IL	Observed robot actions BC, diffusion or VLA policy	High-quality contact-rich data
Human video / retargeting	Human hand or object motion	Embodiment gap and physical contact
Planning + fixed grasp	Geometry and motion constraints	Feasibility under a rigid hand–tool relation
Task-specific sim-to-real RL	Physics, a task model and reward	Object modeling and per-task training

SimToolReal combines human object-motion goals with a reusable controller learned in simulation.

Imitation learning needs the right action data

Robot action supervision

Teleoperation supplies examples of what this robot should do.

- Hand kinematics differ across embodiments.
- Limited force feedback makes dexterous contacts difficult to demonstrate.

Human video supervision

A person can demonstrate the desired tool motion naturally.

- Human finger motion need not transfer to robot fingers.
- Kinematic similarity does not guarantee a stable grasp.

Can we transfer the desired object motion while letting the robot discover its own contact strategy?

Reinforcement learning shifts the engineering cost

Why simulate?

- Physics exposes contact consequences.
- Parallel trials explore grasp and reorientation strategies.
- No robot action demonstrations are required for policy training.

Why is reuse difficult?

- A new object may need a new simulation asset.
- A new task may need a new reward.
- A specialist can overfit to one trajectory.

The paper asks whether many tool tasks can share one training objective.

The transferable interface is the tool motion

TASK SPECIFICATION

Human tool motion

Object pose waypoints



Reusable goal-reaching
RL policy



ROBOT EXECUTION

Arm + hand actions

Specify where the tool goes

Human-to-robot task description

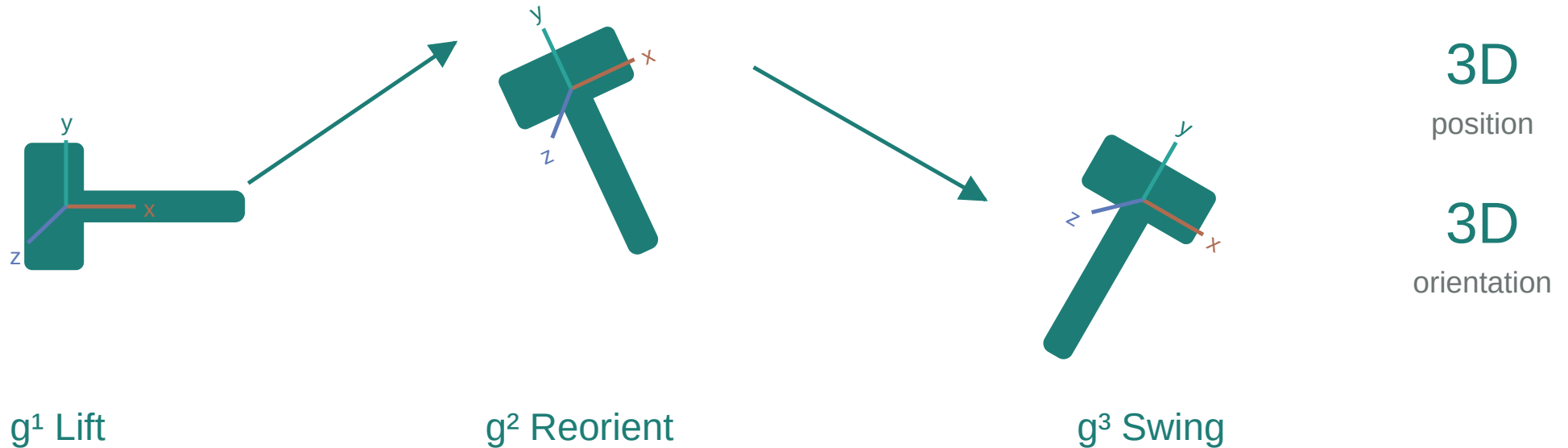
Learn how this robot moves it

Contacts and regrasping emerge from RL

Object goals specify the motion. RL discovers robot-specific grasps and in-hand actions.

A 6D pose specifies position and orientation

A task becomes an ordered set of rigid-object configurations



$$g^k = (R^k, p^k) \in SE(3), \quad \mathcal{G} = \{g^1, \dots, g^K\}$$

A single policy repeatedly solves goal reaching

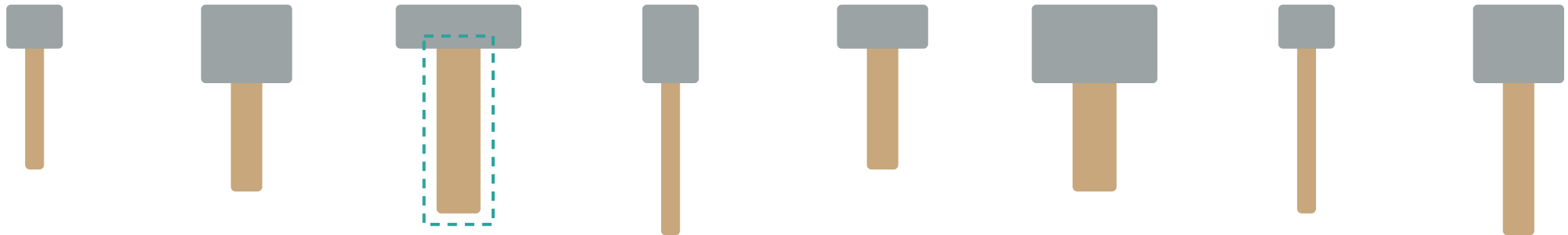
$$a_t = \pi_{\theta}(s_t, o_t, \phi, g^k)$$

Input	Meaning
s_t	Robot proprioception
o_t	Estimated current tool pose
ϕ	Coarse grasp-region bounding box
g^k	Current desired tool pose

Reach the current pose, advance the goal, and reuse the same policy.

Common structure is a designed prior

One structural prior, many physical instances



Geometry

Shape + size

Mass distribution

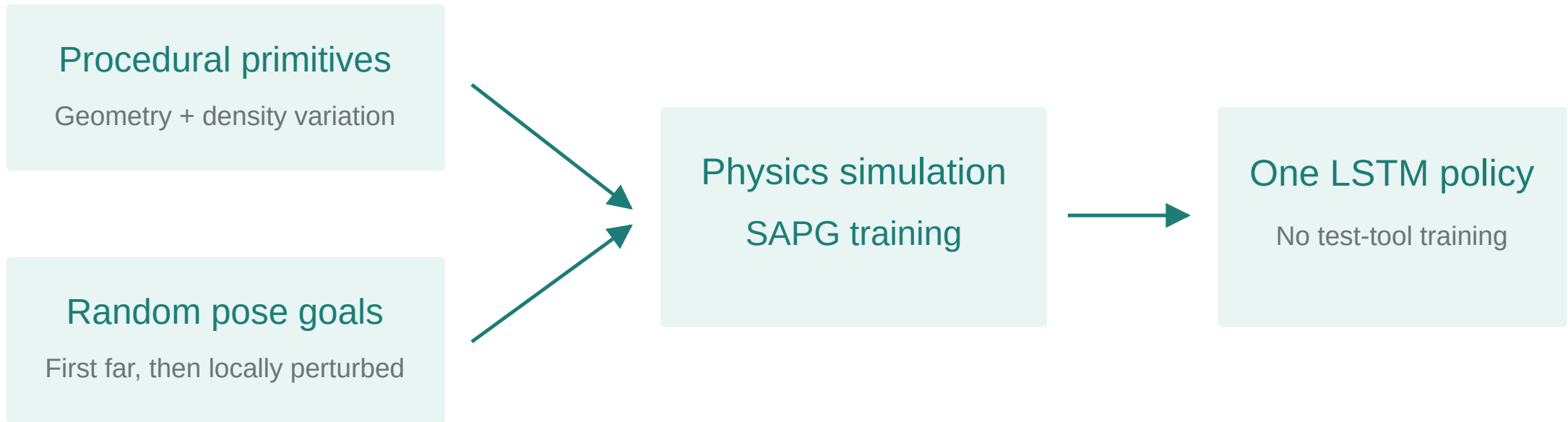
Handle / head densities

Shared policy inputs

Pose + grasp-region box

The commonality is a geometric interface: tool pose and a graspable region.

Training varies objects and goals together



The policy must grasp from the table before it can earn repeated goal-reaching rewards.

A shared reward induces the component skills

$$r = r_{\text{smooth}} + r_{\text{grasp}} + \mathbb{I}_{\text{grasped}} r_{\text{goal}}$$

01 Approach and lift

Shaping helps the hand acquire the object from the table.

02 Reach a pose

Reward improvement in pose distance and give a success bonus.

03 Repeat

After success, sample another goal with the same reward.

“Universal objective” still includes engineered grasping and smoothness terms.

Learning and transfer need more than object diversity

Learning in simulation

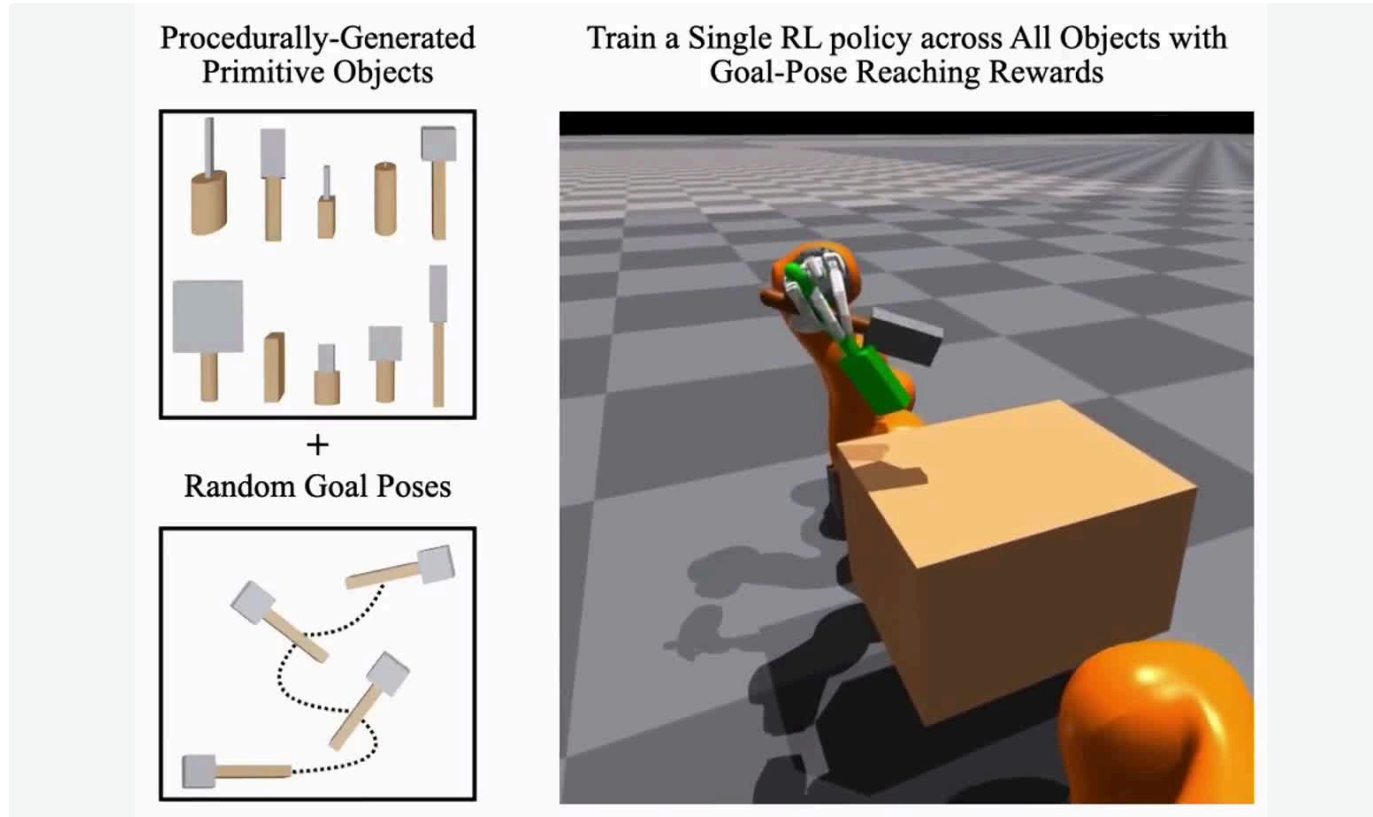
- SAPG improves exploration over the tested PPO setup.
- An asymmetric critic uses privileged simulator state.
- An LSTM actor handles partial observations.

Deploying in reality

- Randomize pose noise and sensing delays.
- Perturb the object with forces and torques.
- Match robot dynamics and smooth the joint targets.

The deployed actor only sees signals that the real system can provide.

Training on synthetic tools produces dexterous motion



WHAT TO WATCH

The **green mesh** indicates the current target pose.

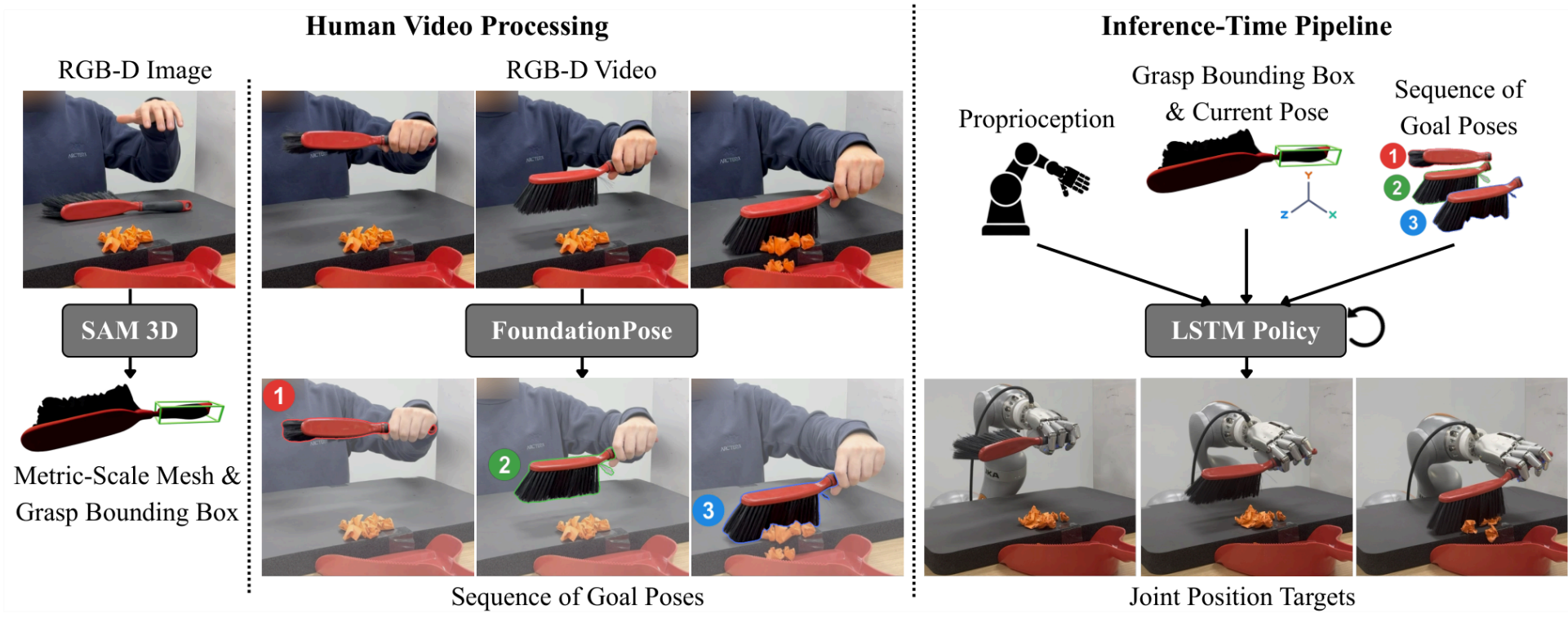
The policy learns to acquire and reorient procedurally generated tools.

There is no human task trajectory in this training loop.

SimToolReal project: training environment · 1× speed · silent excerpt

[Original project video ↗](#)

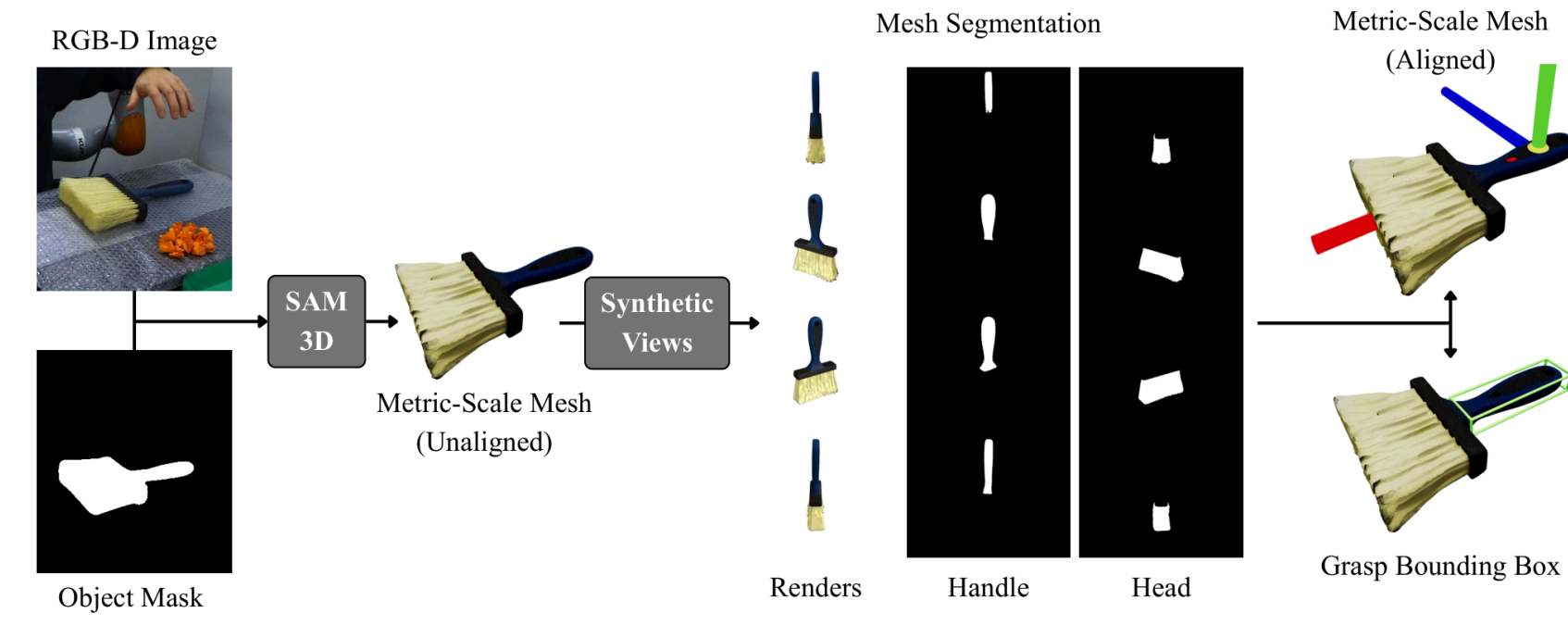
Deployment adds perception and a human demonstration



Paper Fig. 3 · Human video processing and online policy execution

The policy is fixed. A new demonstration supplies tool geometry and goal poses.

Geometry extraction is semi-automated

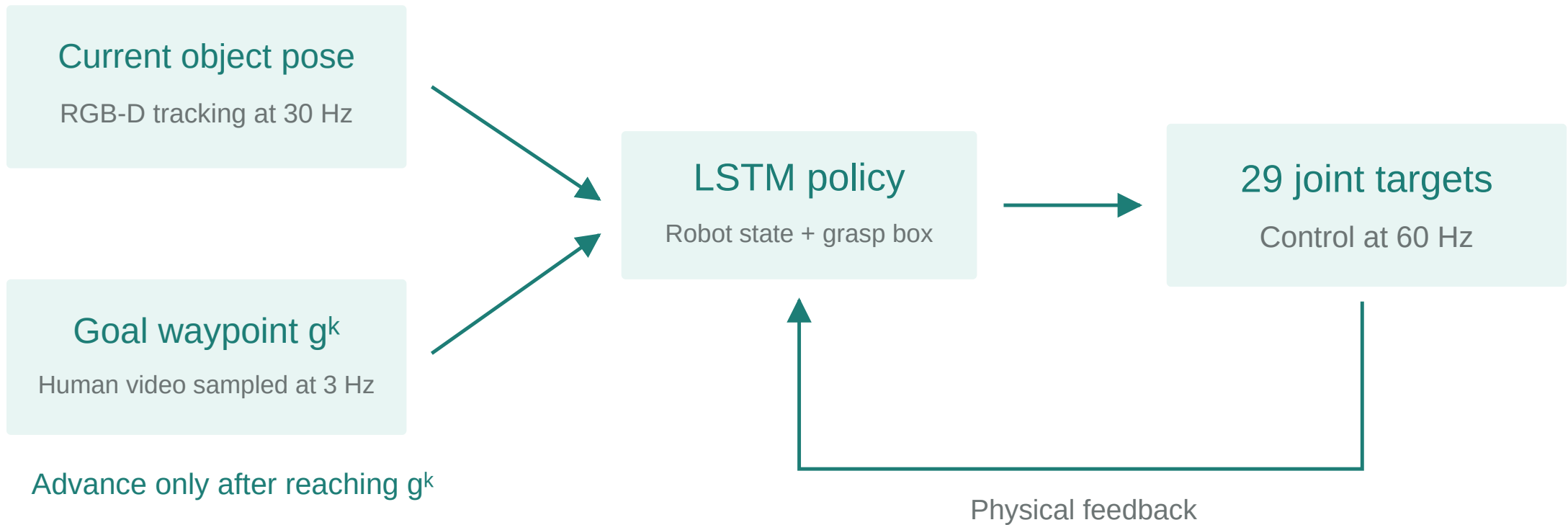


Paper Fig. 12 · Metric mesh, handle/head segmentation and grasp box

SAM 2 uses user point prompts. SAM 3D uses captured depth for metric scale.

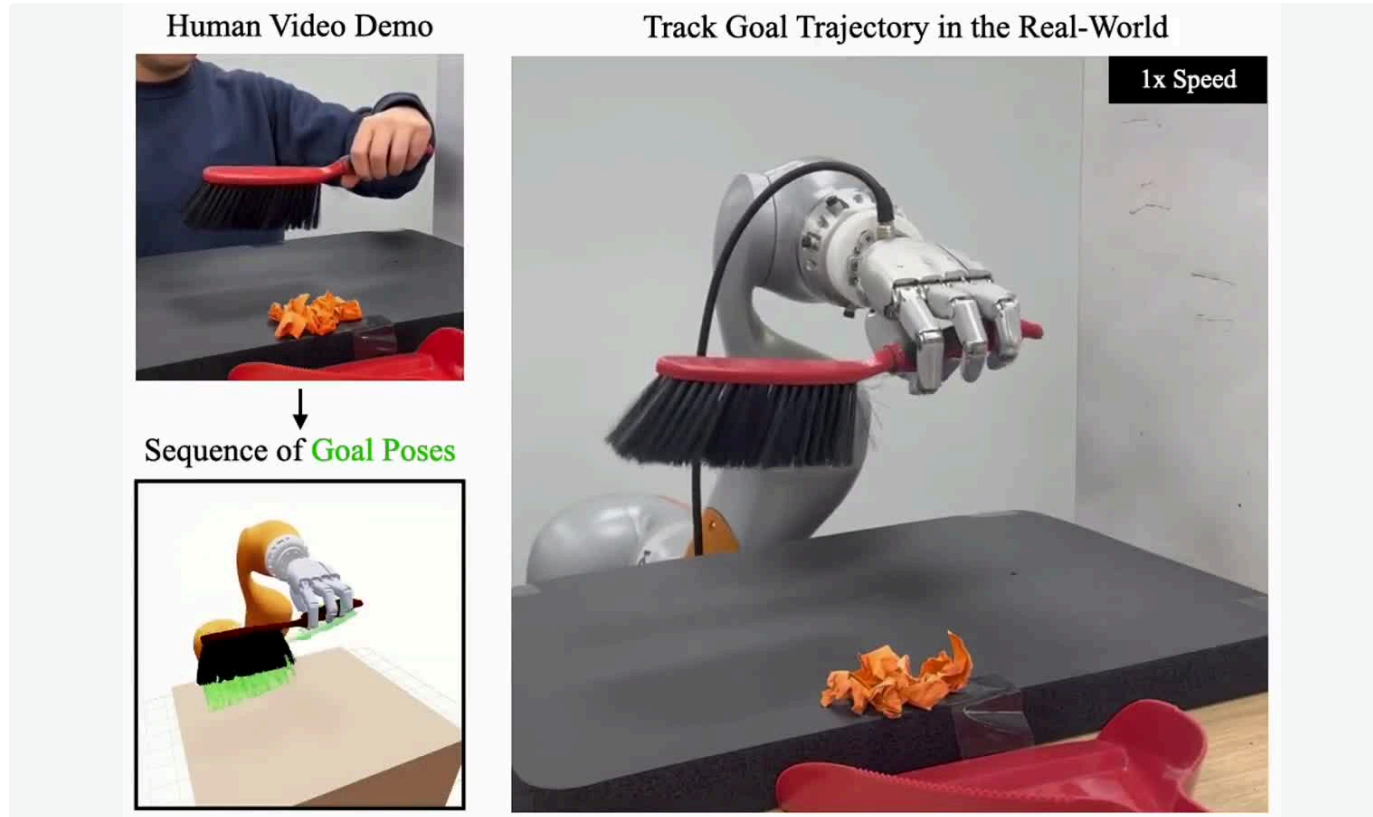
The handle defines the grasp box and canonical frame. FoundationPose then tracks the tool.

Goal switching is event-driven



3 Hz describes demonstration downsampling, not the policy control frequency.

The closed loop follows unseen tool trajectories



SimToolReal project: real-world inference · 1x speed · silent excerpt

[Original project video ↗](#)

WHAT TO WATCH

The human demonstration supplies the **tool-motion goals**.

The fixed policy grasps, reorients and moves the real brush.

The goal interface specifies tool motion rather than robot finger trajectories.

What “zero-shot” means in this paper

Absent at deployment	Still required at deployment
Policy training on the target tool	A metric-scale mesh for pose tracking
Policy training on the target task	An RGB-D human demonstration
Task-specific reward tuning	A grasp region and canonical frame
Real-world policy fine-tuning	Online object tracking and calibrated robot sensing

Zero-shot refers to policy transfer across target objects and trajectories.

DexToolBench evaluates trajectory progress

6

tool categories

12

object instances

24

object–task variations

120

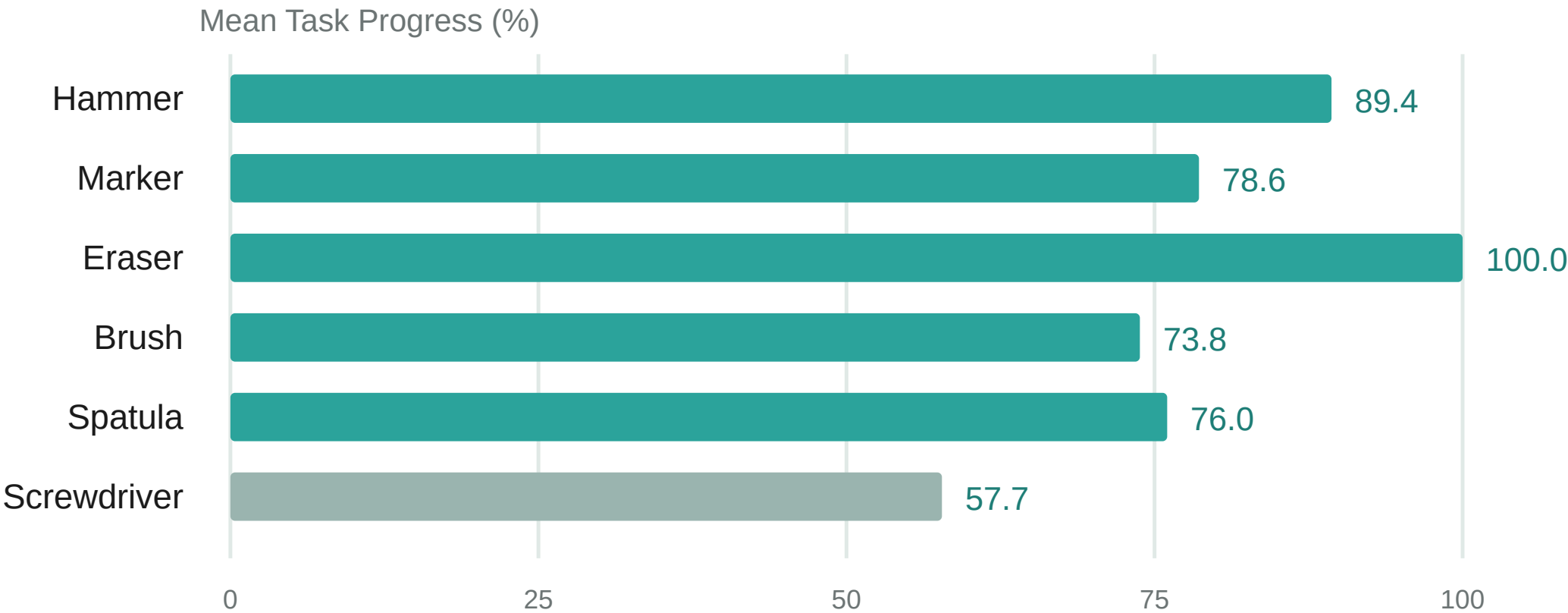
real-world rollouts

$$\text{Task Progress} = \frac{\text{goal poses reached}}{\text{goal poses in the sequence}} \times 100\%$$

The 2 cm tolerance is a maximum keypoint-distance criterion, jointly reflecting translation and rotation.

A completed trajectory does not necessarily imply successful physical task completion.

Transfer is strong but uneven across tools



Presenter aggregation of Table II: **79.3% mean Task Progress** across 120 rollouts. Each category has 20 rollouts.

In-hand control matters in the brush comparison

No initial tool rotation

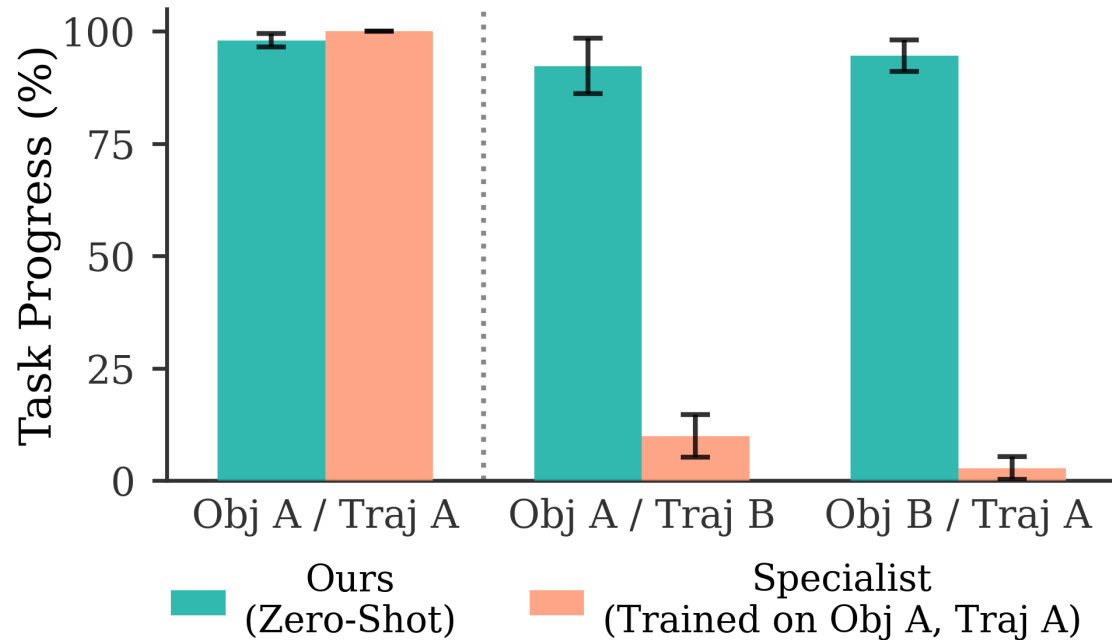


90° tool rotation required



Five rollouts per variation. Same red-brush task, two initial orientations. Values from Fig. 5.

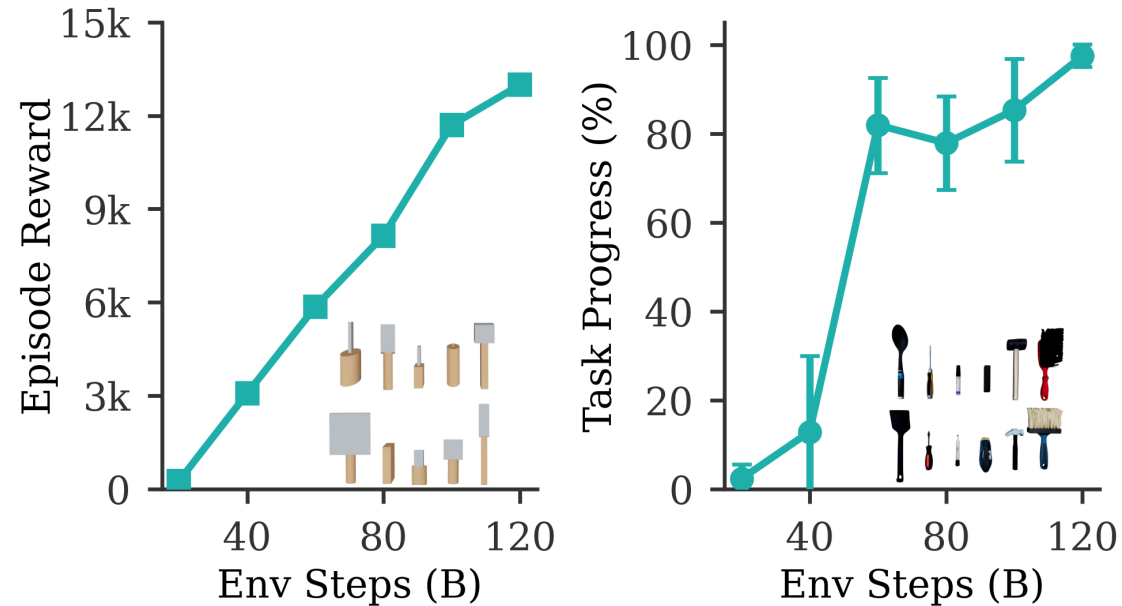
Specialists are sensitive to changed tools and goals



Paper Fig. 6 · Comparison to specialist RL policies

- Match specialist performance on its training setup.
- Keep high progress when either the tool or the trajectory changes.
- All three conditions are evaluated in simulation.

The proxy objective predicts downstream progress



Paper Fig. 7 · Training reward and unseen-task evaluation

Evidence for the abstraction

Random-pose training improves alongside unseen tool-trajectory performance.

Support for the proxy objective, with no guarantee of functional task success.

Perception is the largest recorded failure category



■ Pose tracking loss

43.7%

■ Object drops

34.5%

■ Incomplete rotation

18.2%

■ Grasp failure

3.6%

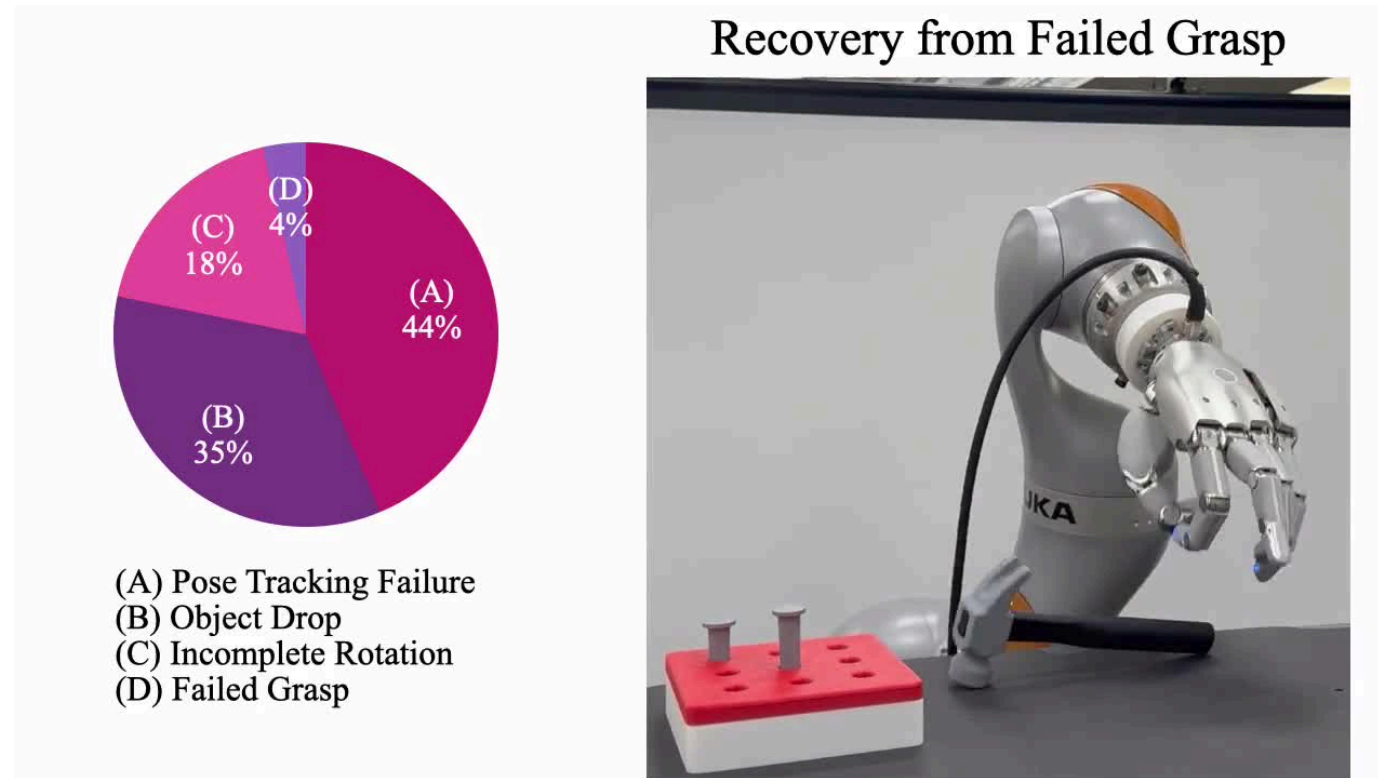
43.7%

of observed failures

Perception is a system bottleneck

A robust controller cannot compensate for a tracker that loses the object entirely.

Recovery depends on retaining usable state feedback



SimToolReal project: recovery behavior · 1× speed · silent excerpt

[Original project video ↗](#)

WHAT TO WATCH

The policy can attempt to regrasp after a manipulation error.

Recovery requires the object to remain reachable and trackable.

This is a qualitative behavior, not an evaluated universal recovery guarantee.

The pose interface leaves important state out

Assumption or omission	Where it can fail
Rigid tool state	Scissors and articulated tools need internal joint state.
Environment-blind policy inputs	Clutter can make a pose path collide with the scene.
No explicit task-force objective	Correct tool motion may not deliver the required force.
Fixed high-level goal sequence	The system does not replan the task when the world changes.

The demonstrated result is reusable trajectory execution under a structured deployment pipeline.

A complementary route beyond end-to-end action models

HYPOTHESIS 1 · PRESENTER DISCUSSION

What this work establishes

A reusable motor policy can emerge from structured simulation and transfer to new tools without policy fine-tuning.

What remains open

Autonomous task discovery, scene understanding and reliable adaptation in unstructured environments.

A plausible route toward in-the-wild deployment: structured RL skills composed by a high-level system.

This paper neither demonstrates online in-the-wild skill learning nor eliminates foundation models.

Other interactions may admit compact task interfaces

HYPOTHESIS 2 · PRESENTER DISCUSSION

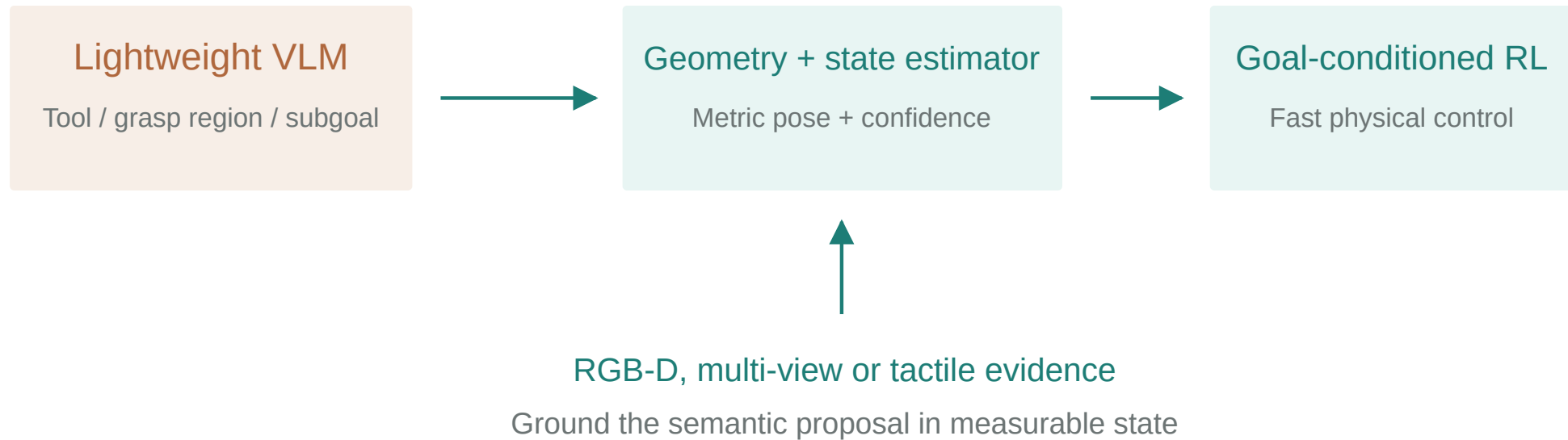
Interaction family	Candidate task interface	Extra state likely needed
Rigid object transport / reorientation	Object poses in SE(3)	Obstacle and contact geometry
Doors, drawers and other mechanisms	Articulation state or constrained pose path	Joint constraints and contact mode
Insertion or forceful surface work	Pose plus contact / wrench targets	Force, compliance and environment state
Deformable-object manipulation	Shape or task-relevant keypoints	Deformation and contact history

The research question is which compact state is sufficient for each interaction family.

A lightweight VLM could improve the semantic interface

HYPOTHESIS 3 · PROPOSED ARCHITECTURE

PROPOSED EXTENSION



A semantic model may reduce prompting and improve goal selection. Metric, low-latency tracking still needs validation.

Three experiments would test these extensions

Proposed change	Controlled comparison	Measure beyond Task Progress
VLM-assisted semantic setup	Manual point prompts vs. VLM region proposals	Human intervention count, grasp-region accuracy
Better closed-loop state estimation	Original tracker vs. multi-view / grounded hybrid	Pose error, latency, tracking-loss rate
Richer task interface	Pose-only vs. pose + contact / force + replanning	Functional completion, contact safety, transfer

Keep the motor policy fixed first to separate perception gains from controller changes.

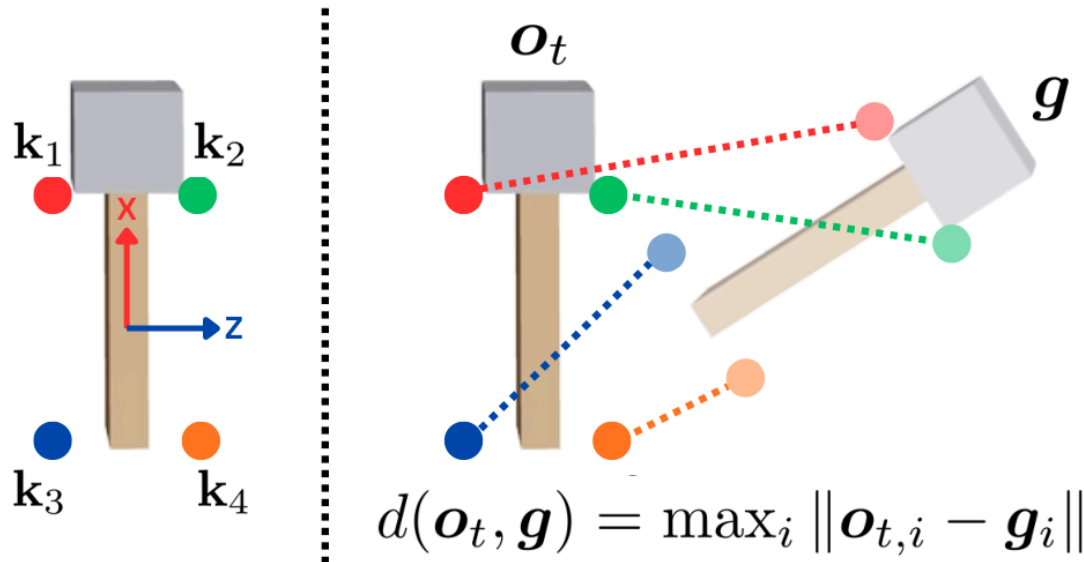
A reusable motor skill needs a well-chosen task interface

- 01 Specify motion at the object level**
Human video supplies goals without prescribing robot finger motion.
- 02 Learn the controller over varied physics**
Procedural primitives and random goals support policy reuse.
- 03 Preserve the boundary of the evidence**
Strong trajectory transfer still leaves perception, force and replanning open.

Which parts of a manipulation task can we safely compress into a reusable goal interface?

BACKUP

Pose error couples translation and rotation



Paper Fig. 9 · Four object-frame pose keypoints

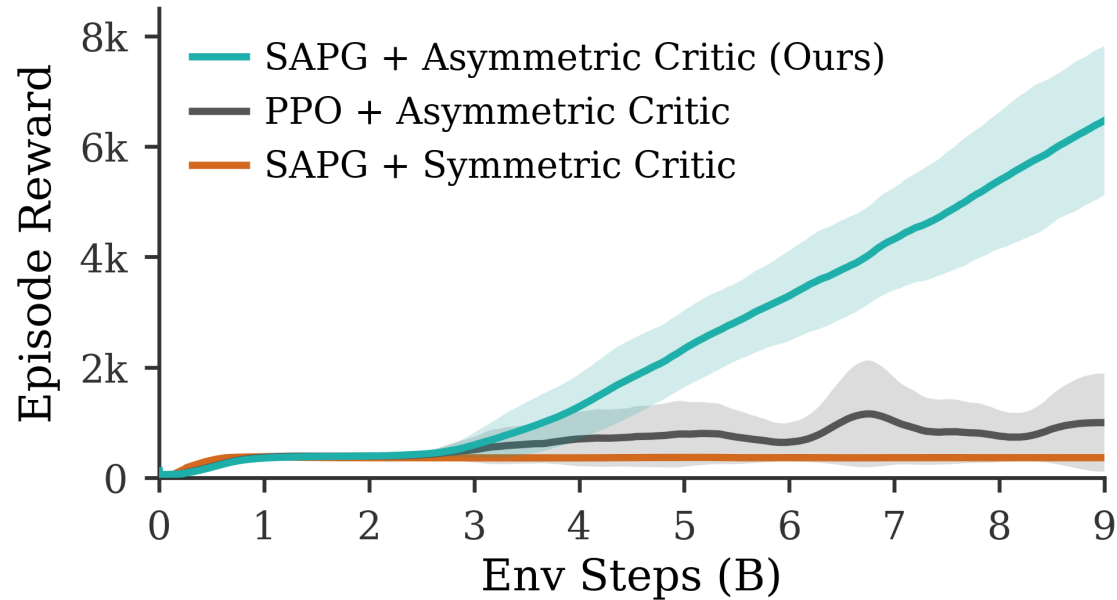
$$d(o_t, g) = \max_{i=1, \dots, 4} \|o_{t,i} - g_i\|$$

- Reward keypoints use fixed scales across objects.
- Observation keypoints depend on the object instance.
- Training tolerance: 1 cm. Evaluation: 2 cm.

A “2 cm pose error” is a keypoint-space criterion, not a separate orientation threshold.

BACKUP

Ablations identify important RL ingredients



Paper Fig. 8 · Training reward, mean over five seeds

- Replacing SAPG with PPO lowers training reward.
- Removing the asymmetric critic severely hinders learning.
- These curves cover 9B environment steps.

This ablation measures training reward, not real-world functional task completion.

Implementation details that matter for reproduction

Component	Reported configuration
Robot	7-DoF KUKA iiwa 14 + 22-DoF Sharpa left hand
Simulator / parallelism	IsaacGym / 24,576 environments
Simulation / control	120 Hz / 60 Hz
Actor	LSTM[1024] + MLP[1024, 1024, 512, 512]
Critic	MLP[1024, 1024, 512, 512], privileged simulator state
Pose observations	1 cm translation noise, 5° rotation noise, up to 10-step delay
Perturbations	5 N force scale, 0.5 Nm torque scale

Real-world task breakdown: Hammer, marker and eraser

Tool / instance	Trajectory	Mean progress
Hammer / Claw	Swing down	100.0%
Hammer / Claw	Swing side	95.5%
Hammer / Mallet	Swing down	84.4%
Hammer / Mallet	Swing side	77.5%
Marker / Sharpie	Draw smile	80.0%
Marker / Sharpie	Write C	77.6%
Marker / Staples	Draw smile	90.6%
Marker / Staples	Write C	66.2%
Eraser / Handle	Wipe smile	100.0%
Eraser / Handle	Wipe C	100.0%
Eraser / Flat	Wipe smile	100.0%
Eraser / Flat	Wipe C	100.0%

BACKUP

Real-world task breakdown: Brush, spatula and screwdriver

Tool / instance	Trajectory	Mean progress
Brush / Blue	Sweep forward	51.1%
Brush / Blue	Sweep right	100.0%
Brush / Red	Sweep forward	82.7%
Brush / Red	Sweep right	61.4%
Spatula / Spoon	Serve plate	85.0%
Spatula / Spoon	Flip over	95.5%
Spatula / Flat	Serve plate	77.5%
Spatula / Flat	Flip over	46.1%
Screwdriver / Long	Spin vertical	55.8%
Screwdriver / Long	Spin horizontal	61.7%
Screwdriver / Short	Spin vertical	37.9%
Screwdriver / Short	Spin horizontal	75.6%

References, videos and source material

Main paper

Kedia, Lum, Bohg and Liu. *SimToolReal: An Object-Centric Policy for Zero-Shot Dexterous Tool Manipulation*.
arxiv.org/abs/2602.16863v2 · [RSS 2026 proceedings](#)

Project and code

simtoolreal.github.io · github.com/tylerlum/simtoolreal

Closely related work

Chen et al. *Object-Centric Dexterous Manipulation from Human Motion Data*. CoRL 2024.

Lum et al. *Crossing the Human-Robot Embodiment Gap with Sim-to-Real RL Using One Human Demonstration*. 2025.

Petrenko et al. *DexPBT*. 2023.

Design reference

[Mizoreww/steerable-vla-talk](https://mizoreww.github.io/steerable-vla-talk) and the previous A2A seminar deck.